

Thesis progress report

LLM-Guided Neural Network Compilation for Real-time UAV Edge Inference

Atul Kumar Gupta · 25MCSS06

The language model may only name a plan

The companion computer still needs a compile recipe. The question here is who may choose it.

Asked to make a network fast, a language model will name passes that do not exist, and it will quote a speed it never measured. Either mistake is enough to make the result unusable in a thesis. A quoted frame rate that was not timed is not a result. A pass with no implementation is not a compiler.

The model is therefore limited to one job. It may name a plan, and the plan must be taken from a fixed list. It may not edit the ONNX file. It may not fly the aircraft. Three other parts do the rest. A checker accepts the plan or cuts it. An engine rewrites the graph. A profiler measures time and compares the new outputs with the untouched network.

The plan arrives as JSON and is checked before any rewrite. If the reply contains a made-up speed, that phrase is removed. Every stored run records that no frame rate was claimed. The separation is the research claim of this step: the model proposes, and only the measured run is evidence.

The tempting design was to let the model rewrite ONNX, or to let it pick TensorRT settings in free text. A trial prompt did what free text always does: it named a SiLU fusion that this CPU cannot run, and it offered a frame rate. That single trial killed the free-text design. The fixed list is not a lack of ambition. It is the only way a later table can say what was tried.

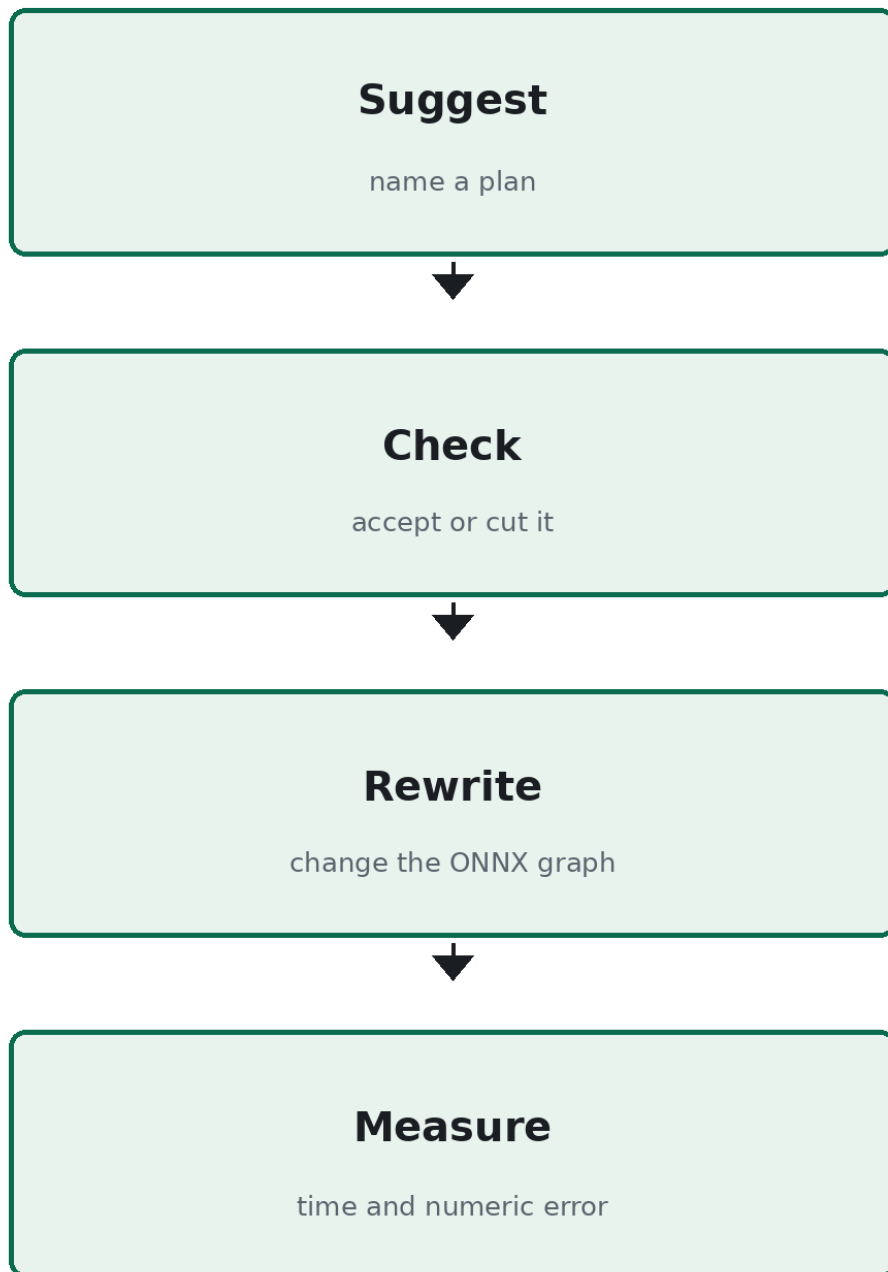


Figure 1. Suggest, check, rewrite, then measure. The model stops at the first box.

The model is the planner. Whether the rewrite actually changes a graph is a separate question.